

Gastronome Créatif - Résumé complet de la séance

Séance du 7 octobre 2026 - ASP.NET Core MVC, Entity Framework Core et PostgreSQL

1. Connexion ASP.NET Core MVC - PostgreSQL

Nous avons mis en place la communication entre le projet ASP.NET Core MVC et PostgreSQL avec Entity Framework Core et le provider Npgsql. La chaîne de connexion a été configurée dans appsettings.json et GCContext a été enregistré dans Program.cs.

```
builder.Services.AddDbContext<GCContext>(options =>
    options.UseNpgsql(
        builder.Configuration.GetConnectionString("GCCConnection")
    ));
```

2. Création et utilisation de GCContext

GCContext représente le lien entre les classes C# et les tables de la base. Nous avons notamment déclaré les DbSet nécessaires pour Categories, Recettes et Categories_recettes.

```
public DbSet<Categories> Categories { get; set; }
public DbSet<Recettes> Recettes { get; set; }
public DbSet<Categories_recettes> Categories_recettes { get; set; }
```

3. Correspondance modèles C# / tables PostgreSQL

Nous avons vu l'utilité de [Key] pour indiquer une clé primaire et de [Table("categories")] pour faire correspondre explicitement une classe C# avec une table PostgreSQL. Nous avons également corrigé le namespace manquant du modèle Recettes en ajoutant namespace GC_MVC.Models.

```
[Table("categories")]
public class Categories
{
    [Key]
    public int id_categorie { get; set; }
    public string nom { get; set; }
}
```

4. Table d'association categories_recettes

Nous avons étudié la relation plusieurs-à-plusieurs entre categories et recettes. La table categories_recettes possède une clé primaire composée de id_categorie et id_recette. Cette configuration a été déclarée dans OnModelCreating.

```
modelBuilder.Entity<Categories_recettes>()
    .HasKey(cr => new
    {
        cr.id_categorie,
        cr.id_recette
    });
```

5. Affichage des catégories

La page /Categories affiche désormais les catégories provenant de PostgreSQL. Nous avons revu la différence entre @model, qui déclare le type attendu par une vue Razor, et @Model, qui représente l'objet réellement reçu par la vue.

```
@model List<GC_MVC.Models.Categories>

@foreach (GC_MVC.Models.Categories categorie in Model)
```

```
{  
    ...  
}
```

6. Catégories cliquables et paramètres de route

Chaque catégorie a été transformée en lien Razor. asp-controller choisit le contrôleur, asp-action choisit l'action et asp-route-id transmet l'identifiant de la catégorie. MVC réalise ensuite le model binding vers le paramètre int id.

```
<a asp-controller="Categories"  
    asp-action="Recettes"  
    asp-route-id="@categorie.id_categorie">  
    @categorie.nom  
</a>
```

7. Recherche des recettes d'une catégorie

Nous avons d'abord filtré la table d'association avec Where et ToList, puis réalisé une jointure LINQ entre recettes et categories_recettes afin d'obtenir directement les recettes appartenant à la catégorie sélectionnée.

```
List<Recettes> recettes =  
(  
    from recette in _context.Recettes  
    join association in _context.Categories_recettes  
    on recette.id_recette equals association.id_recette  
    where association.id_categorie == id  
    select recette  
) .ToList();
```

8. Transmission de la liste à la vue

Le contrôleur transmet List avec return View(recettes). La vue Views/Categories/Recettes.cshtml est fortement typée et utilise Model.Count et foreach pour afficher les recettes.

```
@model List<GC_MVC.Models.Recettes>  
  
<p>Nombre de recettes : @Model.Count</p>  
  
@foreach (GC_MVC.Models.Recettes recette in Model)  
{  
    <p>Titre : @recette.nom</p>  
}
```

9. Affichage dynamique des informations et des images

Nous avons affiché le nom, les temps de préparation et de cuisson, la difficulté, le créateur et la photo. Le nom du fichier image vient de PostgreSQL et est combiné au chemin de wwwroot.

```

```

10. Réutilisation de la maquette de fiche recette

La maquette HTML/CSS déjà préparée pour une recette sera conservée et transformée progressivement en vue Razor dynamique. Le header et le footer ne seront pas dupliqués puisqu'ils sont déjà centralisés dans _Layout.cshtml. La maquette contient notamment le titre, la photo, les informations principales, les ingrédients, la préparation, les étapes, les notes et commentaires.

11. Création du lien vers la fiche détaillée

Chaque recette de la liste est maintenant cliquable. Son id_recette est transmis à l'action Detail du RecettesController.

```
<a asp-controller="Recettes"  
    asp-action="Detail"
```

```
asp-route-id="@recette.id_recette">
    @recette.nom
</a>
```

12. RecettesController et action Detail

Nous avons créé RecettesController avec injection de GCContext. L'action Detail reçoit l'identifiant de la recette sélectionnée.

```
public IActionResult Detail(int id)
{
    ...
}
```

13. Recherche d'une recette avec FirstOrDefault

Contrairement à la page de catégorie, nous voulons ici un seul objet Recettes et non une liste. FirstOrDefault recherche la première recette dont id_recette correspond à l'id reçu. Comme aucune recette peut être trouvée, la variable est nullable avec Recettes?.

```
Recettes? recette = _context.Recettes
    .FirstOrDefault(recette => recette.id_recette == id);
```

14. Gestion du cas null et de la 404

Si FirstOrDefault ne trouve aucune recette, recette vaut null. Nous avons donc ajouté une vérification qui renvoie NotFound().

```
if (recette == null)
{
    return NotFound();
}
```

15. Création de Views/Recettes/Detail.cshtml

Nous avons appliqué la convention MVC : RecettesController + action Detail conduit naturellement à Views/Recettes/Detail.cshtml. Cette vue reçoit une seule recette, donc son modèle n'est pas List mais Recettes.

```
@using GC_MVC.Models
@model Recettes

<h1>@Model.nom</h1>
```

16. Diagnostic et correction de l'erreur 500

Une erreur 500 avec NullReferenceException apparaissait lors de l'affichage de Detail.cshtml. Les logs montraient que la requête SQL fonctionnait correctement. Le problème venait du contrôleur : return View() ouvrait la vue sans lui transmettre la recette. La correction a été de retourner View(recette).

```
return View(recette);
```

17. État du parcours MVC à la fin de la séance

Le parcours fonctionnel est maintenant : PostgreSQL -> catégories -> clic sur une catégorie -> jointure categories_recettes/recettes -> liste de recettes -> clic sur une recette -> RecettesController.Detail(id) -> FirstOrDefault -> objet Recettes -> Detail.cshtml -> affichage de @Model.nom.

18. Prochaine étape

La prochaine séance reprendra depuis Detail.cshtml. La maquette existante sera rendue dynamique progressivement : d'abord les données déjà disponibles dans Recettes (nom, photo, temps_preparation, temps_cuisson, difficulté, createur), puis les relations vers l'utilisateur/auteur, les ingrédients et quantités, les

étapes, et enfin les avis, notes et commentaires. La progression restera volontairement incrémentale : explication, petit exercice, essai, puis correction.

Notions principales travaillées : MVC, conventions de vues, Entity Framework Core, DbSet, clés primaires et clés composées, LINQ, Where, ToList, jointures, FirstOrDefault, nullable/null, routes, model binding, vues Razor fortement typées, @model/@Model, foreach, namespaces, fichiers statiques et diagnostic d'une erreur 500.